FICHA TÉCNICA — Reto 2 · Infraestructura web segura (LEMP)

Nginx + PHP-FPM 8.5 + MariaDB sobre Ubuntu Server 26.04 LTS · Equipo: Juan Diego López, Luis Camilo Vivas Cruz, Dylan Mancipe · Sistemas Operacionales 2026-2, Universidad El Bosque

1. ¿Qué problema de negocio resuelve?

Publicar una aplicación web que guarda datos en una base de datos sin dejarla expuesta: que solo el personal autorizado pueda modificar la información, que el resto solo pueda consultarla, que todo viaje cifrado y que un problema en una aplicación no afecte a la otra que vive en el mismo servidor.

2. Glosario

Término	Definición
Virtual host	Bloque *server* de Nginx. Un mismo servidor e IP atiende varios sitios; Nginx elige el bloque según el nombre pedido (*server_name*).
PHP-FPM / pool	Servicio que ejecuta PHP fuera del servidor web. Un pool es un grupo de procesos con su propio usuario y su propio socket.
FastCGI	Protocolo con el que Nginx le entrega a PHP-FPM la petición para que ejecute el *.php* y le devuelva el resultado.
Socket Unix	Archivo especial (p. ej. */run/php/…sock*) para comunicar dos procesos del mismo equipo; tiene dueño, grupo y permisos como cualquier archivo.
TLS / certificado autofirmado	TLS cifra la conexión (HTTPS). Autofirmado = cifra igual, pero ninguna autoridad lo respalda: el navegador advierte.
Redirección 301	Respuesta “se movió permanentemente”: el puerto 80 manda al cliente a la versión *https://*.
Mínimo privilegio	Dar a cada usuario solo lo que necesita: *web_read* solo SELECT; si intenta escribir, MariaDB responde error 1142.
502 Bad Gateway	Nginx (el intermediario) no obtuvo respuesta válida del servicio de atrás (PHP-FPM). 504 = tardó demasiado; 500 = falló la aplicación.

3. Arquitectura

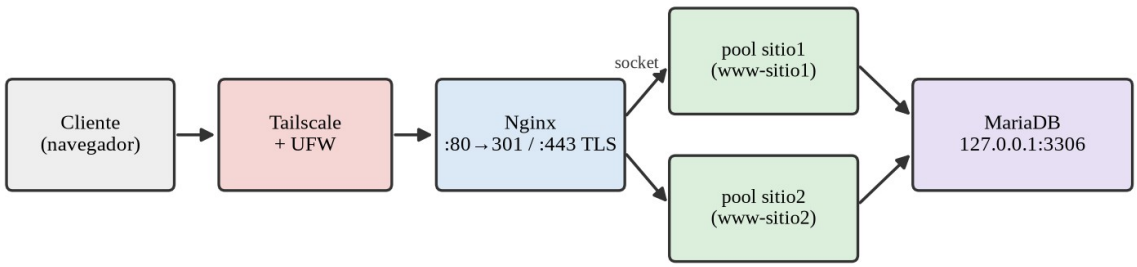


Figura 1. Flujo de una petición en la solución del equipo.

El cliente resuelve *sitio1.local*/*sitio2.local* con */etc/hosts* y llega al servidor por la VPN Tailscale; el firewall UFW solo acepta 22/80/443 si entran por esa interfaz. Nginx responde 301 en el puerto 80 y, en el 443, termina TLS, elige el vhost por el nombre y envía los *.php* por FastCGI al socket del pool de ese sitio. Cada pool corre con su propio usuario y consulta MariaDB, que solo escucha en 127.0.0.1.

4. Decisión de diseño clave: Nginx frente a Apache

Nginx atiende miles de conexiones con pocos procesos (modelo de eventos) y siempre ejecuta PHP aparte, con PHP-FPM; Apache crea procesos o hilos por conexión y suele ejecutar PHP dentro de sí mismo (mod_php, todo con el mismo usuario) y lee *.htaccess* en cada petición. Ganó Nginx porque al usar PHP-FPM pudimos dar a cada sitio su propio usuario y socket (aislamiento), porque toda la configuración está centralizada y se valida con *nginx -t*, y por su bajo consumo en una VM de 2 GB. Apache conviene cuando cada usuario necesita configurar su carpeta con *.htaccess*.

5. Configuración crítica explicada

- **fastcgi_pass unix:/run/php/php8.5-fpm-sitio1.sock;** (vhost de Nginx): a dónde manda Nginx los *.php*. Si apunta a un socket que no existe o sin permiso, el sitio da 502.
- **listen.owner / listen.group = www-data** (pool de PHP-FPM): quién puede conectarse al socket. Debe ser el usuario de Nginx (*www-data*). No confundir con *user/group*, que es quién ejecuta el PHP del sitio.

- **return 301 https://$host$request_uri; y server_tokens off;** (Nginx): el primero obliga a usar HTTPS sin servir nada por HTTP; el segundo oculta la versión de Nginx en encabezados y páginas de error, para no regalar información a un atacante.

6. Caso de estudio: el incidente

Síntoma	Causa	Solución
sitio1 responde 200 y sitio2 responde 502 Bad Gateway. PHP-FPM aparece activo (hipótesis "servicio caído" descartada). *sitio2.error.log*: connect() … sitio2.sock failed (13: Permission denied).	En el pool de sitio2 se puso *listen.owner/group = www-sitio2*: el socket quedó a nombre del usuario del pool, con modo 0660, y Nginx (*www-data*) ya no podía escribir en él. errno 13 = sin permiso (errno 2 sería socket inexistente).	Devolver *listen.owner/group = www-data*, validar con *php-fpm8.5 -t*, reiniciar *php8.5-fpm* y verificar: ambos sitios 200 y el monitor en RESUMEN=OK.

7. Buenas prácticas

- Validar antes de recargar: *nginx -t*, *php-fpm8.5 -t*, *sshd -t*; y probar el comportamiento, porque una configuración sintácticamente válida puede estar mal.
- Un pool PHP-FPM por sitio, con usuario propio y *open_basedir*: una falla o intrusión en un sitio no alcanza al otro.
- Firewall por interfaz (solo *tailscale0*) y MariaDB en 127.0.0.1: lo que no está expuesto no se puede atacar.
- Antes de tocar SSH o el firewall, confirmar el acceso por llave en una sesión nueva y tener una consola de respaldo.
- Leer el *error.log* del sitio antes de reiniciar servicios: el número de error dice dónde buscar.

8. Preguntas de repaso

- Si un sitio PHP responde 502 y en el log aparece "(2: No such file or directory)", ¿qué revisarías primero y en qué cambia si dijera "(13: Permission denied)"?
- ¿Qué diferencia hay entre *user* y *listen.owner* en un pool de PHP-FPM, y cuál de los dos aísla un sitio de otro?
- ¿Por qué el usuario *web_read* recibe el error 1142 al hacer un INSERT, y en qué capa se aplica ese control?